СВОДКА ПРОЕКТА · ОКТЯБРЬ 2026

Компас

Цифровой ассистент врача и пациента для ведения тревожных и депрессивных состояний. Рабочий прототип (веб-приложение) на синтетических данных для кейс-чемпионата «Клинический компас».

TypeScript · React 18 · Vite 5 · CSS · JSON-правила · работает в браузере без сервера

1 Если коротко

Что это
Сайт с тремя кабинетами: пациента (с телефона), врача (с ноутбука) и терапевта. Всё, что отмечает пациент, сразу видно врачу.

Что делает
Считает опросники, проверяет препарат по инструкции и рекомендациям Минздрава, следит за приёмом лекарства, вовремя подаёт сигнал врачу.

Чего не делает
Не ставит диагноз и ничего не назначает: решает врач. У каждого вывода есть кнопка «Почему?» — правило и источник.

2 Технологии

ЧТО	ЧЕМ СДЕЛАНО	ЧТО ПОГУГЛИТЬ
Язык	**TypeScript 5.6** (строгий режим `strict`) — JavaScript со статическими типами. Компилируется в JavaScript (ES2021).	TypeScript handbook
Интерфейс	**React 18**, компоненты в файлах `.tsx` (синтаксис JSX), функциональные компоненты и хуки (`useState`, `useEffect`).	React hooks, JSX
Навигация	**React Router 6**, режим `HashRouter`: адреса вида `#/doctor`, `#/patient` — работает с любого хостинга и из файла.	react-router HashRouter
Стили	Обычный **CSS** с переменными (дизайн-токены: цвета, шрифты, отступы). Без Tailwind и Bootstrap. Шрифты Golos Text и Manrope.	CSS custom properties
Сборка	**Vite 5** (сервер разработки и сборка), **Node.js** 20+ и **npm**. Офлайн-версия одним HTML-файлом — плагин `vite-plugin-singlefile`.	Vite guide, npm scripts
Правила и данные	Файлы **JSON**: паспорт препарата, правила сигналов, маршрутизация, шкалы, источники. Код ядра их только исполняет — «правила как данные».	rule engine, rules as data
Хранение	IndexedDB браузера в зашифрованном виде (**Web Crypto**, AES-GCM 256); вкладки синхронизируются через `BroadcastChannel`. Сервера в прототипе нет.	Web Crypto AES-GCM, BroadcastChannel
Графики	Свои компоненты на **SVG**, без библиотек графиков.	SVG charts
Распознавание текста	**tesseract.js 6** (WebAssembly) прямо в браузере, модели русского и английского языков. Сканы никуда не отправляются.	tesseract.js, OCR WebAssembly
Голосовой ввод	Диктовка — **Web Speech API**; голосовые сообщения — **MediaRecorder**, с расшифровкой.	Web Speech API, MediaRecorder
Тесты	**Vitest 2** — 120 автотестов ядра и сценариев. **Playwright** (`playwright-core`) с установленным Edge — 27 проверок в браузере и скриншоты.	Vitest, Playwright
Прочее	Иконки `lucide-react`, QR-коды `qrcode.react`. Публикация — **GitHub Pages** через **GitHub Actions**.	GitHub Pages Actions

3 Как устроено

Кабинет пациента
Телефон: отметка приёма, самочувствие за минуту, опросники, чат, «Мне тяжело сейчас».

→

«Сервер» стенда + клиническое ядро
`src/store/api.ts` — точка записи; ядро `src/core` исполняет JSON-правила: шкалы, противопоказания, сигналы, маршрут.

→

Кабинет врача
Ноутбук: сигналы по срочности, досье «на 30 секунд», терапия и безопасность, сообщения, журнал.

Данные лежат в браузере и мгновенно расходятся по открытым вкладкам. Для пилота `api.ts` заменяется клиентом настоящего сервера (REST API, например FastAPI или NestJS + PostgreSQL) — интерфейс и ядро не меняются.

4 Где что лежит

ПАПКА	ЧТО ВНУТРИ
`src/core`	Клиническое ядро без интерфейса, чистые функции: движок правил, подсчёт шкал, приверженность, разбор текста, дозировки, график врача.
`src/data`	Правила как данные (JSON): `packs/aviandr.json` — паспорт препарата, `alertRules.json` — 17 правил сигналов, `routingRules.json`, `scales.json`, `sources.json`.
`src/store`	«Сервер» стенда: API действий, генератор синтетических пациентов, 7 демо-сценариев, синхронизация вкладок. Здесь же автотесты.
`src/patient` `src/doctor` `src/therapist`	Экраны трёх ролей: пациент (мобильный формат), врач (рабочее место на ноутбуке), терапевт (скрининг и маршрут к специалисту).
`src/pages` `src/ui`	Главная, демо-сценарии, соответствие кейсу; общие компоненты (графики, окна, кнопка «Почему?», голосовой ввод, распознавание).
`scripts`	Скрипты Node.js: скриншоты, проверки в браузере, офлайн-копия, модели распознавания.
`docs`	`PLAN.md` — план и статус, `screens/` — 43 скриншота; описание — `README.md`.

5 Принципы

- **Врач решает.** Система предлагает, считает и напоминает, но ничего не назначает сама.
- **Правила как данные.** У правила есть код, версия и источник; исполняются только утверждённые.
- **Объяснимость.** «Почему?» показывает правило, входные данные и источник.
- **Без внешних языковых моделей.** Одинаковые данные — одинаковый результат.
- **Только синтетические данные.** Прототип не медицинское изделие.

6 Термины предметной области

СППВР (CDSS)	поддержка врачебных решений
PHQ-9, GAD-7, PHQ-4	опросники депрессии и тревоги
HAM-D, HAM-A, MADRS	шкалы, которые заполняет врач
Приверженность	регулярность приёма препарата
МКБ-10 (F41.1)	классификатор диагнозов
КР, ГРЛС	рекомендации; реестр лекарств
ИДС	добровольное согласие пациента
152-ФЗ, 323-ФЗ	о персональных данных; о здоровье

7 Как запустить

```
# нужен Node.js 20 или новее
npm install            # зависимости
npm run dev            # сайт: localhost:5180
npm test               # 120 автотестов
npm run build          # сборка в dist/
npm run build:offline  # один HTML-файл
```

8 В цифрах

≈14 400	строк кода TypeScript / TSX
120	автотестов + 27 проверок в браузере
7	демо-сценариев с эталонными проверками
10	синтетических пациентов на стенде
17	правил сигналов трёх уровней срочности
0,9 МБ	офлайн-версия одним HTML-файлом